

**REPORT ON STUDENT INDUSTRIAL WORK EXPERIENCE SCHEME
(SIWES)**

**EXPERIENCES AND MINI PROJECT UNDERTAKEN AT
START INNOVATION HUB, UYO, AKWA IBOM STATE**

ON

**NEWS IN MAIL: A PERSONALISED NEWS BRIEFING SYSTEM WITH
SCHEDULED EMAIL DELIVERY AND AI SUMMARIES**

BY

FRIDAY, GODSWILL ESSIEN

23/SC/CO/158

**DEPARTMENT OF COMPUTER SCIENCE, FACULTY OF COMPUTING
UNIVERSITY OF UYO, UYO, AKWA IBOM STATE**

COURSE CODE: CSC 329

**SUBMITTED TO: DEPARTMENT OF COMPUTER SCIENCE, FACULTY OF
COMPUTING, UNIVERSITY OF UYO, UYO, AKWA IBOM STATE**

**IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF A
BACHELOR OF SCIENCE (B.SC) DEGREE IN COMPUTER SCIENCE**

OCTOBER, 2026

CERTIFICATION

This is to certify that this report on the Student Industrial Work Experience Scheme (SIWES) undertaken at Start Innovation Hub, Uyo, Akwa Ibom State, was carried out and written by FRIDAY GODSWILL ESSIEN with registration number 23/SC/CO/158, of the Department of Computer Science, Faculty of Computing, University of Uyo, and has been approved as meeting the requirements of the course CSC 329.

.....
Industry-Based Supervisor, Start Innovation Hub

.....
Date

.....
Departmental SIWES Coordinator

.....
Date

.....
Head of Department, Computer Science

.....
Date

DEDICATION

I dedicate this report to God Almighty, the source of my life, strength and wisdom, and to my parents and family, whose love, prayers and support carried me through my industrial training.

ACKNOWLEDGEMENT

My first thanks go to God Almighty for His grace, protection and guidance from the first day of my industrial training to the completion of this report.

I am grateful to the management of Start Innovation Hub, Uyo, and to its Chief Executive Officer, Mr Hanson Johnson, for accepting me as an intern and giving me real work. I sincerely thank my industry supervisor and the developers of the Software Development unit, who patiently taught me, reviewed my code and trusted me with real projects. I also thank the training team for allowing me to assist at classes and at the Kids Bootcamp, and my fellow interns for the shared challenges and the friendly competition that pushed all of us to do better.

I appreciate the Head of Department, my departmental SIWES coordinator and all the lecturers of the Department of Computer Science, University of Uyo, for the knowledge that prepared me for this training and for their guidance on this report. I also thank the Industrial Training Fund for organising the scheme.

Finally, I am deeply thankful to my parents and family for their encouragement, prayers and financial support throughout my training.

ABSTRACT

This report presents my experience during the Student Industrial Work Experience Scheme (SIWES) at Start Innovation Hub, Uyo, Akwa Ibom State, from April to September 2026, where I was attached to the Software Development unit. It begins with a history of SIWES and a profile of Start Innovation Hub, a technology hub founded in 2014 that has trained more than 4,000 young people in digital skills, supported 13 startups and builds websites and applications for clients.

My work centred on web and backend development. I built strong foundations in HTML, CSS, JavaScript, TypeScript, React and Next.js, then focused on backend development: REST APIs with Node.js and Express, PostgreSQL database design and SQL, input validation, authentication with hashed passwords and signed tokens, and security practices. I worked in a team with Git and pull requests, built an event registration API, contributed to a client website, and assisted instructors during training sessions.

For my mini project I designed and built News in Mail, a web application that lets readers choose topics and delivery times and then sends them a personalised news briefing every morning and evening. The system collects stories from 20 free RSS feeds, picks fresh stories for each reader without repeats, summarises them with a large language model through OpenRouter, and records every briefing as sent, failed or skipped. A test email is sent as soon as a reader finishes onboarding. Built with Next.js, TypeScript, Drizzle ORM and PostgreSQL, the system passed 77 automated tests and is deployed on Railway.

TABLE OF CONTENTS

CERTIFICATION.....	i
DEDICATION.....	ii
ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
LIST OF FIGURES.....	vii
LIST OF TABLES.....	viii
CHAPTER ONE: BRIEF HISTORY OF SIWES.....	1
1.1 Introduction.....	1
1.2 Brief history of SIWES in Nigeria.....	1
1.3 Vision and mission of SIWES.....	2
1.4 Aims and objectives of SIWES.....	2
1.5 Benefits and problems of SIWES.....	2
CHAPTER TWO: BRIEF HISTORY OF THE SIWES ORGANISATION.....	3
2.1 Introduction.....	3
2.2 History and milestones.....	3
2.3 Services of Start Innovation Hub.....	4
2.4 Training programmes and impact.....	4
2.5 Organisational structure.....	5
2.6 SIWES students at the Hub.....	5
2.7 Vision, mission and core pillars.....	5
CHAPTER THREE: EXPERIENCE GAINED AND WORK DONE.....	6
3.1 Introduction.....	6
3.2 Orientation and development environment.....	6
3.3 Frontend foundations.....	7
3.4 How the web works.....	7
3.5 Backend development with Node.js.....	8
3.6 Databases and SQL.....	8
3.7 Authentication and security.....	9
3.8 Version control and teamwork.....	9
3.9 Project experience 1: event registration API.....	10
3.10 Project experience 2: client website in Next.js.....	10
3.11 Challenges faced.....	10
3.12 Conclusion.....	10
CHAPTER FOUR: MINI PROJECT: NEWS IN MAIL.....	11
4.1 Introduction.....	11
4.2 Statement of the problem.....	11
4.3 Aim and objectives.....	11
4.4 Scope and significance.....	11
4.5 Review of related concepts.....	12
4.5.1 RSS Feeds.....	12

4.5.2 Large Language Models for Summarisation.....	12
4.5.3 Scheduling and Time Zones.....	12
4.6 Methodology.....	12
4.7 Requirements analysis.....	12
4.8 System design.....	13
4.8.1 System Architecture.....	13
4.8.2 The Briefing Process.....	14
4.8.3 Sequence of the Test Email.....	15
4.8.4 Database Design.....	15
4.9 Implementation.....	16
4.10 User interface.....	17
4.11 Testing.....	21
4.12 Deployment.....	22
4.13 Results and discussion.....	22
4.14 Conclusion.....	23
CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS.....	24
5.1 Summary.....	24
5.2 Conclusion.....	24
5.3 Recommendations.....	24
REFERENCES.....	25
APPENDIX: LIVE SYSTEM AND SOURCE CODE.....	26

LIST OF FIGURES

Figure 2.1: Key milestones of Start Innovation Hub.....	3
Figure 2.2: Simplified organisational structure of Start Innovation Hub.....	5
Figure 3.1: My six-month learning and work path.....	6
Figure 3.2: The request and response cycle of a web application.....	7
Figure 3.3: The layered structure I used for backend APIs.....	8
Figure 3.4: The feature-branch workflow used by the team.....	9
Figure 4.1: The four development increments.....	12
Figure 4.2: Use case diagram of News in Mail.....	13
Figure 4.3: System architecture.....	14
Figure 4.4: Flowchart of the briefing process.....	14
Figure 4.5: Sequence diagram for onboarding and the test email.....	15
Figure 4.6: Entity relationship diagram.....	15
Figure 4.7: Landing page.....	17
Figure 4.8: Onboarding: picking topics and delivery times.....	18
Figure 4.9: The test email shown right after onboarding.....	18
Figure 4.10: Home page with top stories, next email and recent emails.....	19
Figure 4.11: Inbox showing sent, failed and skipped briefings.....	19
Figure 4.12: A failed briefing with the mail server error.....	20
Figure 4.13: Settings, with pause and resume.....	20
Figure 4.14: Mobile view: home (left) and an email (right).....	21
Figure 4.15: Deployment pipeline on Railway.....	22

LIST OF TABLES

Table 1.1: Bodies involved in SIWES and their roles.....	1
Table 1.2: Benefits of SIWES to each party.....	2
Table 2.1: Main services of Start Innovation Hub.....	4
Table 2.2: Reported impact of Start Innovation Hub.....	4
Table 3.1: Summary of my main areas of work.....	6
Table 3.2: HTTP methods and status codes I used most.....	7
Table 3.3: SQL commands I used regularly.....	9
Table 4.1: Functional requirements.....	12
Table 4.2: Non-functional requirements.....	13
Table 4.3: Data dictionary of the digests table.....	15
Table 4.4: Tools and technologies used.....	16
Table 4.5: Summary of automated tests.....	21
Table 4.6: Selected functional test cases.....	22

CHAPTER ONE

BRIEF HISTORY OF SIWES

1.1 INTRODUCTION

The Student Industrial Work Experience Scheme (SIWES) is a compulsory skills training programme for students of Nigerian universities, polytechnics and colleges of education in courses that need practical exposure. For a Computer Science student, it is the period when lessons on programming, databases and networks are tested against real users, real deadlines and real systems that must keep running.

The scheme exists to close the gap between what is taught in class and what is done at work. Students are placed in organisations related to their field, where they learn how work is planned, how teams communicate and how quality is kept. At the University of Uyo, SIWES is taken in the third year as the course CSC 329. My placement ran from April to September 2026 at Start Innovation Hub, a technology hub in Uyo, Akwa Ibom State, where I worked mainly on web and backend development. This report describes the scheme, the organisation, the work I did and the mini project I built at the end of the placement.

1.2 BRIEF HISTORY OF SIWES IN NIGERIA

SIWES was established in 1973 by the Industrial Training Fund (ITF). The ITF was set up by Decree 47 of 1971 to promote the acquisition of skills in industry and commerce and to build a pool of trained indigenous manpower. Employers at the time complained that many graduates, especially in science and engineering, had good certificates but little practical skill.

At first the ITF funded and managed the scheme alone, with a small number of students. As participation grew the cost became too high, and in 1979 the ITF withdrew. The National Universities Commission (NUC) and the National Board for Technical Education (NBTE) managed it for a period, but in 1984 the Federal Government returned it to the ITF with government funding. Since 1985 the ITF has run SIWES together with the NUC, the NBTE and the National Commission for Colleges of Education (NCCE).

Table 1.1: Bodies involved in SIWES and their roles

Body	Main role in SIWES
Federal Government	Funds the scheme and sets national policy on skills development.
Industrial Training Fund (ITF)	Manages the scheme, provides logbooks and forms, pays allowances and monitors placements through its area offices.
NUC, NBTE and NCCE	Supervising agencies that approve qualifying courses and monitor institutions.
Institutions (University of Uyo)	Place students, appoint coordinators and supervisors, and assess logbooks and reports.

Body	Main role in SIWES
Employers (Start Innovation Hub)	Accept students, assign industry supervisors, give real work and sign the logbook.
Students	Attend regularly, keep an honest logbook, follow workplace rules and write a report.

1.3 VISION AND MISSION OF SIWES

Vision: to be a reliable bridge between classroom learning and the world of work, producing graduates who are skilled, confident and ready to contribute to national development.

Mission: to expose students of Nigerian tertiary institutions to real industrial practice through supervised placements, so that they gain practical skills, good work habits and professional values.

1.4 AIMS AND OBJECTIVES OF SIWES

- To give students the chance to apply their theoretical knowledge to real work.
- To expose students to tools, equipment and methods that may not be available in their institutions.
- To prepare students for the conditions of work they will meet after graduation and ease the move from school to work.
- To teach discipline, punctuality, teamwork and responsibility in a professional setting.
- To strengthen links between employers and institutions so that what is taught stays relevant to industry.

1.5 BENEFITS AND PROBLEMS OF SIWES

Table 1.2: Benefits of SIWES to each party

Party	How it benefits
Student	Gains practical skills and a portfolio of real work. In my case this meant APIs, websites, databases and a deployed web application.
Institution	Gets feedback on how relevant its curriculum is and produces more employable graduates.
Employer	Gets extra hands for real projects and a chance to assess possible future staff.
Nation	Builds a skilled indigenous workforce for the digital economy.

The scheme also has problems. Students often struggle to find placements that match their course, some organisations give students only errands, allowances are often delayed and transport is costly. I was fortunate because Start Innovation Hub trains developers every day and gave me structured, hands-on work. I describe the challenges I faced in Section 3.11.

CHAPTER TWO

BRIEF HISTORY OF THE SIWES ORGANISATION

Start Innovation Hub, Uyo

2.1 INTRODUCTION

Start Innovation Hub (popularly called Starthub) is an information and communication technology company and innovation lab in Uyo, Akwa Ibom State. It was officially incorporated in May 2014 and was founded by Mr Hanson Johnson, who has been active in the Uyo technology community since about 2011 and remains its Chief Executive Officer. The hub's purpose is to grow tech talent and help technology startups with software and hardware ideas start up faster.

The hub first operated from Suites 304 and 305 of the Ibom e-Library complex on IBB Avenue, Uyo, and now operates from Plot 6, Unit C, Ewet Housing Estate, Uyo. For more than ten years it has partnered with international and national bodies to train young people in digital skills, run incubation programmes for founders and build websites and applications for clients.

2.2 HISTORY AND MILESTONES

In its early years the hub partnered with Google and trained close to 500 software developers. Some of them were deployed as facilitators in the Andela Learning Community programme powered by Google. Later partners included Facebook, Microsoft, the World Bank, the United Nations Development Programme (UNDP), the National Information Technology Development Agency (NITDA), the PIND Foundation and the Akwa Ibom State Government.

In January 2025, while marking ten years of operation, the CEO announced a plan to support ideation projects that could attract at least 100 million US dollars in funding for startups in Akwa Ibom State, with a focus on venture building for the next ten years. In July 2025 the hub committed 40 million naira to scholarships and discounts for digital skills training. It also announced partner recognition from the Federal Government and approval by the NBTE as a skills training centre. Figure 2.1 summarises these milestones.



Figure 2.1: Key milestones of Start Innovation Hub

2.3 SERVICES OF START INNOVATION HUB

Table 2.1: Main services of Start Innovation Hub

Service	Description
Business Catalyst	Incubation and acceleration for early-stage founders: mentorship, help with business models and pitches, product testing, investor introductions and seed funding.
Web Design and Software Development	Websites and web and mobile applications built for businesses and individuals. This is the unit where I spent most of my placement.
TalentPort Academy	Training and job placement in web development, UI/UX design, data analysis, cybersecurity, Python and machine learning.
Co-working space	Desks, meeting space and fast internet for freelancers, startups and remote workers.

2.4 TRAINING PROGRAMMES AND IMPACT

Training is the best-known part of the hub's work. Courses are project-based and taught by practising developers and designers, so trainees leave with a portfolio. Besides the main courses, the hub runs several community programmes:

- Leavers Trybe: digital skills training for school leavers through TalentPort Academy.
- CLAP (Code Like A Pro): coding, STEM and robotics for secondary school students.
- Kids Bootcamp Uyo 2026: a holiday bootcamp introducing children to coding and digital creativity.
- Starthub Women: programmes that train women in ICT skills and support women in business.
- Makers Club and short masterclasses, for example a Microsoft Excel masterclass.
- Partner programmes such as the World Bank IDEAS project with the Federal Ministry of Education, the UNDP and European Union Nigeria Jubilee Fellows Programme and the 3 Million Technical Talent (3MTT) programme.

Table 2.2: Reported impact of Start Innovation Hub

Measure	Figure reported by the hub
Young people trained in digital skills	More than 4,000, about 60% connected to jobs
Young women trained in ICT skills	More than 1,000
Youths trained in the Niger Delta region	More than 660
Startups supported (and jobs created by 2023)	13 startups, 157 jobs
Scholarships and discounts committed in 2025	40 million naira

2.5 ORGANISATIONAL STRUCTURE

The hub is led by the Chief Executive Officer. The public team page lists a Head of Training, a Business Development Manager and a Program Manager, supported by instructors, developers, administrative staff and interns. Figure 2.2 is a simplified chart based on the public team page and on what I observed. The Software Development unit, where I was attached, is highlighted.

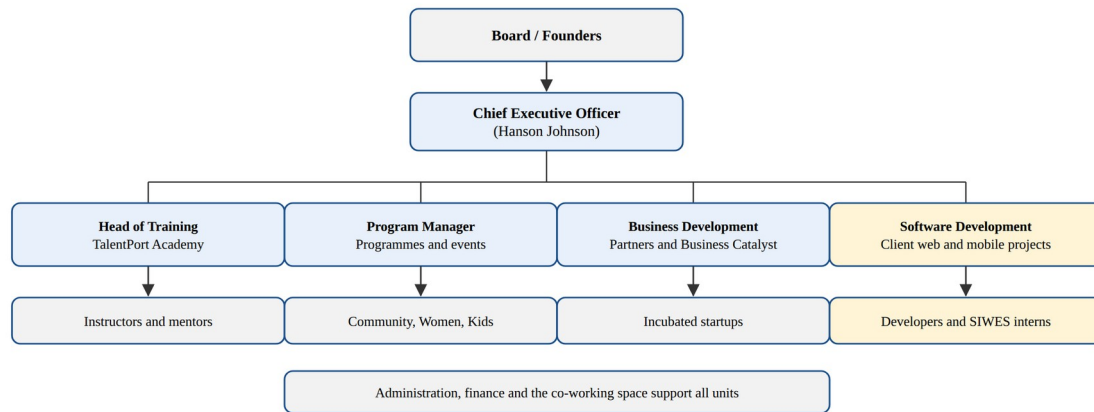


Figure 2.2: Simplified organisational structure of Start Innovation Hub

2.6 SIWES STUDENTS AT THE HUB

Start Innovation Hub regularly takes SIWES and IT students from universities and polytechnics in Akwa Ibom State and nearby states. Computer Science, Computer Engineering and related students are placed in software development, where they go through an intensive learning track and then join real projects. They work on client websites and internal tools, help as assistants at bootcamps and community events, and present their work to the team. Interns use the same co-working space as founders and full-time developers, so they also learn by watching how startups and professionals work.

2.7 VISION, MISSION AND CORE PILLARS

Vision: to be the leading catalyst for technological advancement and innovation across industries.

Mission: to accelerate the development of tech talents and innovative startups by providing them with the necessary tools, mentorship and support.

Core pillars: Innovation, Empowerment and Community.

CHAPTER THREE

EXPERIENCE GAINED AND WORK DONE

3.1 INTRODUCTION

I was attached to the Software Development unit for six months. My supervisor organised the work in phases. First I built strong foundations in the tools and languages of the web. Then I moved to backend development and databases, worked on team projects, and finally built my mini project. Figure 3.1 shows this path and Table 3.1 lists my main areas of work.

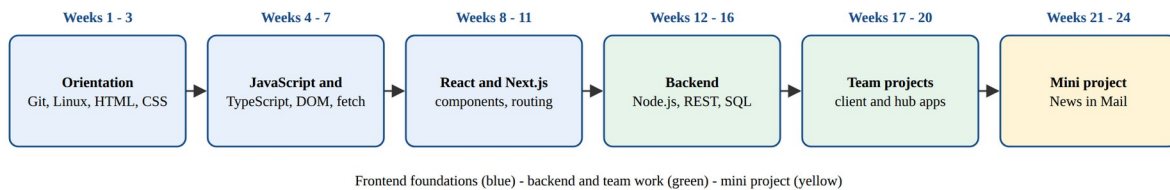


Figure 3.1: My six-month learning and work path

Table 3.1: Summary of my main areas of work

Area	What I did
Development environment	Set up VS Code, Node.js, Git and the terminal; learned the team's coding conventions.
Frontend development	HTML, CSS, JavaScript and TypeScript, then React and Next.js; responsive layouts that work on phones.
Backend development	REST APIs with Node.js and Express, validation, authentication and error handling.
Databases	PostgreSQL design, SQL queries, migrations and the Drizzle ORM.
Teamwork and delivery	Git branches and pull requests, code review, testing and deployment to the cloud.
Training support	Assisted instructors during beginner classes and the Kids Bootcamp.

3.2 ORIENTATION AND DEVELOPMENT ENVIRONMENT

In my first week I was introduced to the units of the hub, the rules of the co-working space and the people I would work with. I installed the tools every developer at the hub uses: Visual Studio Code, Node.js, Git, a PostgreSQL server and the Chrome developer tools. I learned basic Linux commands in the terminal, how to read error messages instead of guessing, and how to search official documentation. The most important habit I picked up was to commit small pieces of work often, with clear messages, so that mistakes are easy to undo.

3.3 FRONTEND FOUNDATIONS

Although my focus was the backend, the hub insisted that every developer should understand the full journey of a web page. I practised semantic HTML (header, nav, main, article and form elements), CSS layout with Flexbox and Grid, and responsive design with media queries. I learned that a grid column should be written as `minmax(0, 1fr)` so long text cannot push a page wider than a phone screen, a lesson I used again in my mini project.

I then learned modern JavaScript: variables and scope, arrays and objects, functions, promises and `async/await`, and the `fetch` API for calling a server. Moving to TypeScript showed me how types catch mistakes before the code runs. Finally I learned React components, props and state, and Next.js routing, server components and server actions. These let one project hold both the pages and the backend logic.

3.4 HOW THE WEB WORKS

Before writing APIs I had to understand the request and response cycle in detail (Figure 3.2). A browser looks up the server's address through DNS, sends an HTTP request, and the server runs code that may read or write the database before it returns HTML or JSON. Table 3.2 lists the HTTP methods and status codes I used most.

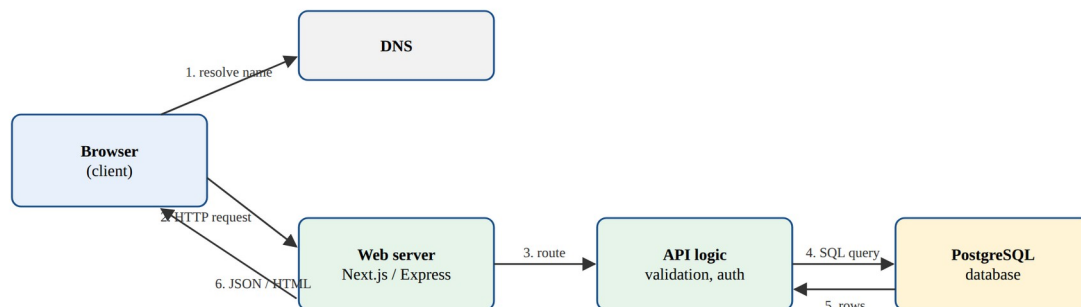


Figure 3.2: The request and response cycle of a web application

Table 3.2: HTTP methods and status codes I used most

Item	Meaning and when I used it
GET	Read data, for example a list of events. It must not change anything on the server.
POST	Create something, for example a new registration or a login.
PATCH / PUT	Update part or all of an existing record.
DELETE	Remove a record.
200 / 201	Success; 201 means a new record was created.
400 / 422	The request was invalid, for example a missing or badly formatted field.
401 / 403	Not signed in, or signed in but not allowed to do this.
404 / 409	Record not found, or a conflict such as a duplicate email.

Item	Meaning and when I used it
500	An unexpected error on the server, which I learned to log and never show in detail to users.

3.5 BACKEND DEVELOPMENT WITH NODE.JS

Backend development took the largest part of my placement. I built REST APIs with Node.js and Express, first with data held in memory and then with a real database. My supervisor taught me to organise every API in layers (Figure 3.3) so that each part has one job: routes map URLs to controllers, middleware validates input and checks the user, controllers handle the request and response, services hold the business rules, and the data layer talks to the database.

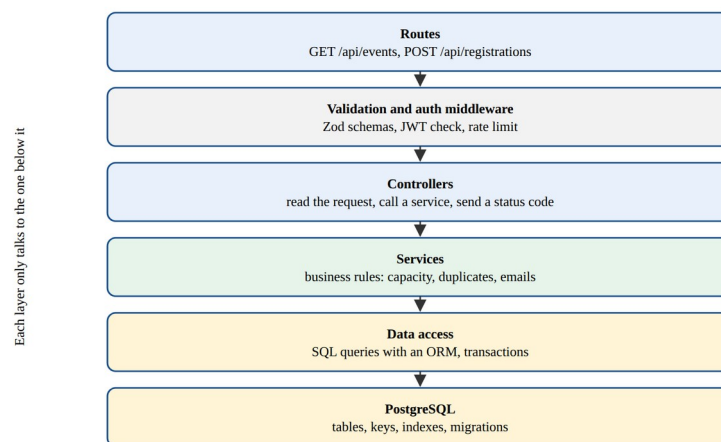


Figure 3.3: The layered structure I used for backend APIs

I validated every input with Zod schemas and returned clear messages for each field instead of a single generic error. I wrote middleware for logging and error handling, used environment variables for secrets instead of hard-coding them, and tested endpoints with Postman and later with automated tests.

3.6 DATABASES AND SQL

I learned to design relational databases in PostgreSQL. This included choosing primary and foreign keys, normalising data so the same fact is not stored twice, and adding indexes and unique constraints so the database itself protects important rules. I wrote SQL by hand before using an ORM, so I would understand what the ORM generates. Table 3.3 shows the commands I used most. I also learned versioned migrations: every change to the schema is saved as a numbered SQL file and applied in order, so every computer and the production server have the same structure.

Table 3.3: SQL commands I used regularly

Command	Purpose
CREATE TABLE	Define a table with columns, types, keys and constraints.
SELECT ... JOIN	Read related data from two tables, for example users and their registrations.
INSERT ... ON CONFLICT	Insert a row but skip or update it if a unique value already exists.
UPDATE / DELETE ... WHERE	Change or remove only the intended rows.
GROUP BY with COUNT	Produce summaries such as registrations per event.
CREATE INDEX / UNIQUE	Speed up searches and stop duplicate records.
BEGIN ... COMMIT	Run several changes as one transaction that succeeds or fails together.

3.7 AUTHENTICATION AND SECURITY

I implemented sign up and sign in several times. Passwords are never stored as plain text; they are hashed with bcrypt. After a successful sign in, the server issues a signed JSON Web Token (JWT) stored in an HTTP-only cookie, so browser scripts cannot read it. Every protected route checks this token, and every query that returns private data is filtered by the signed-in user's id. I also studied the OWASP Top 10 risks and learned to prevent SQL injection with parameterised queries, cross-site scripting by escaping output, and brute-force guessing with rate limits.

3.8 VERSION CONTROL AND TEAMWORK

All work at the hub is kept in Git repositories on GitHub. I followed the feature-branch workflow in Figure 3.4: create a branch for each task, commit in small steps, open a pull request, respond to review comments, and merge only when the automated checks pass. Reviews by senior developers taught me a great deal about naming, simple design and writing code that others can read.

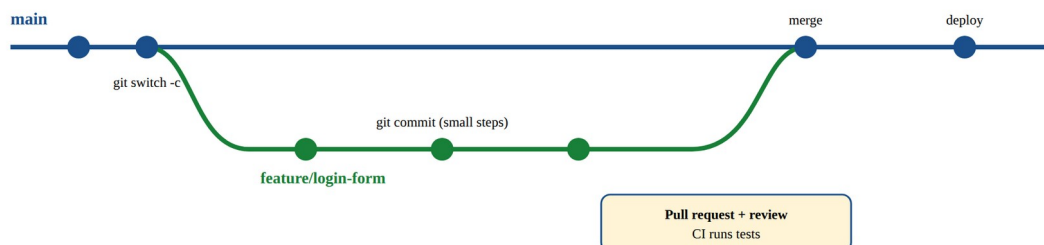


Figure 3.4: The feature-branch workflow used by the team

3.9 PROJECT EXPERIENCE 1: EVENT REGISTRATION API

The hub runs many events: bootcamps, meetups, demo days and masterclasses. As a training project that was later used for some events, I built a registration API with a teammate. The API lists upcoming events, lets participants register with their name, email and phone number, stops registration when an event is full, refuses duplicate registrations with the same email, and lets an administrator export the list of attendees as a CSV file.

We used Node.js, Express, PostgreSQL and Zod. The capacity check and the insert run inside one transaction, so two people cannot take the last seat at the same moment. A unique index on (event, email) blocks duplicates even if the application code has a bug. This was my first time seeing how the database can enforce business rules, and it shaped the design of my mini project.

3.10 PROJECT EXPERIENCE 2: CLIENT WEBSITE IN NEXT.JS

I joined a small team building a website for a client, a local business that wanted a modern site with a contact form and a simple admin page for updating its services. I built the responsive service pages and the contact form. Its server action validates input, saves the message to the database and notifies the client. I also fixed layout problems on small phones and improved page load speed by removing unused images and scripts. At the end I took part in deploying the site and checking it on different devices.

Apart from development, I assisted instructors at beginner HTML and CSS classes and at the Kids Bootcamp. I helped participants set up their computers and found the bugs in their first pages. Explaining ideas in simple words improved my own understanding.

3.11 CHALLENGES FACED

- Erratic power supply and internet at home sometimes delayed my practice work, so I did as much as possible at the hub.
- Transport cost from my residence to Ewet Housing Estate was high relative to the allowance.
- The pace of learning was fast. Backend topics such as transactions and authentication took repeated practice before I was comfortable.
- Debugging deployment errors was difficult at first because the cloud environment behaved differently from my laptop.

3.12 CONCLUSION

My time in the Software Development unit turned me from a student who had read about programming into a developer who can design a database, build and secure an API, work in a team with Git, and deploy a working application. The mini project in the next chapter brings these skills together.

CHAPTER FOUR

MINI PROJECT: NEWS IN MAIL

A Personalised, AI-Summarised News Briefing Service

4.1 INTRODUCTION

News in Mail is a web application where readers choose the subjects they care about, such as Nigeria, World, Business, Technology and Sports, and when they want to read. Every morning and evening the system collects stories from free news sources, picks the most recent and relevant ones for each reader, summarises each story with an AI language model, and sends a short, clean email briefing. A test email is sent as soon as a new reader finishes setting up. The site itself looks and works like Google News, so readers can also browse, search and see every email they have received.

4.2 STATEMENT OF THE PROBLEM

Many young Nigerians follow news through social media, where important stories are mixed with rumours, adverts and posts that are not news. Reading several news websites every day takes time and data, and most sites show every subject even when a reader only cares about a few. Existing email newsletters are usually written by one publisher, cover subjects the reader did not choose, and arrive at times that suit the publisher, not the reader.

4.3 AIM AND OBJECTIVES

The aim of the project is to design and build a web application that delivers a short, personalised and summarised news briefing by email twice a day. The specific objectives are to:

- i. allow readers to create an account and choose topics, delivery times and time zone;
- ii. collect stories automatically from free RSS feeds of trusted news sources;
- iii. summarise each selected story in one or two sentences using a large language model;
- iv. send each briefing once, at the right local time, without repeating stories;
- v. keep a record of every briefing as sent, failed or skipped, and show it in an inbox;
- vi. provide a clean, mobile-friendly interface, test the system thoroughly and deploy it to the cloud.

4.4 SCOPE AND SIGNIFICANCE

The system covers nine topics fed by 20 free RSS feeds from BBC News, The Guardian, Al Jazeera, TechCrunch, Premium Times, Punch, Channels TV and Vanguard. It focuses on Nigerian readers, with Africa/Lagos as the default time zone. It does not write original news; it only summarises and links to the source, so the publishers still receive the full visit. The project

shows how scheduled jobs, external data, AI services and email can be combined safely in one maintainable web application, the kind of product many startups at the hub try to build.

4.5 REVIEW OF RELATED CONCEPTS

4.5.1 RSS Feeds

Really Simple Syndication (RSS) is an XML format that news sites publish so that programs can read their latest stories. Each item has a title, link, publication date and short description. Because it is a public, free and stable format, it is a lawful and reliable way to collect headlines without scraping web pages.

4.5.2 Large Language Models for Summarisation

A large language model (LLM) is a neural network trained on large amounts of text that can follow instructions such as "summarise this in two sentences". OpenRouter is a service that gives one API for many models. The project uses Claude Sonnet 5.5 as the main model and Gemini Flash as an automatic fallback, and it asks the model to reply in JSON so the program can read the answer reliably.

4.5.3 Scheduling and Time Zones

A briefing at 7:00 AM means 7:00 AM where the reader lives. The server runs in UTC, so every check converts the current instant to the reader's local date and time with the Intl API. Each scheduled job must also be idempotent: running it twice must not send two emails.

4.6 METHODOLOGY

I used an iterative and incremental model. The system was built in four increments (Figure 4.1), and each increment was planned, built, tested and reviewed with my supervisor before the next began. This kept a working version available at all times and let me correct the design early.

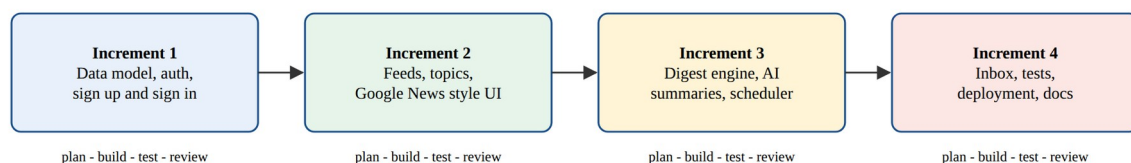


Figure 4.1: The four development increments

4.7 REQUIREMENTS ANALYSIS

Table 4.1: Functional requirements

ID	Requirement
FR1	A reader can sign up, sign in and sign out. The demo account is pre-filled on the sign-in page.
FR2	A new reader picks at least one topic, a morning and/or evening time and a time zone.
FR3	A test email is sent as soon as onboarding is finished.

ID	Requirement
FR4	The system reads all feeds automatically and stores new stories once.
FR5	The system sends each enabled briefing at the reader's local time, at most once per day.
FR6	Stories are summarised by AI, with a fallback summary if the AI service fails.
FR7	The reader can browse topics, search stories, view every briefing and pause or resume.

Table 4.2: Non-functional requirements

Quality	Requirement
Security	Hashed passwords, signed HTTP-only session cookies and every query scoped to the signed-in user.
Reliability	No duplicate emails even if the scheduler runs twice; failures recorded, not hidden.
Usability	Clean Google News style interface that works at phone width with no sideways scrolling.
Maintainability	Business rules in pure, tested functions; versioned database migrations.
Cost	Each story summarised only once and cached, to keep AI cost low.

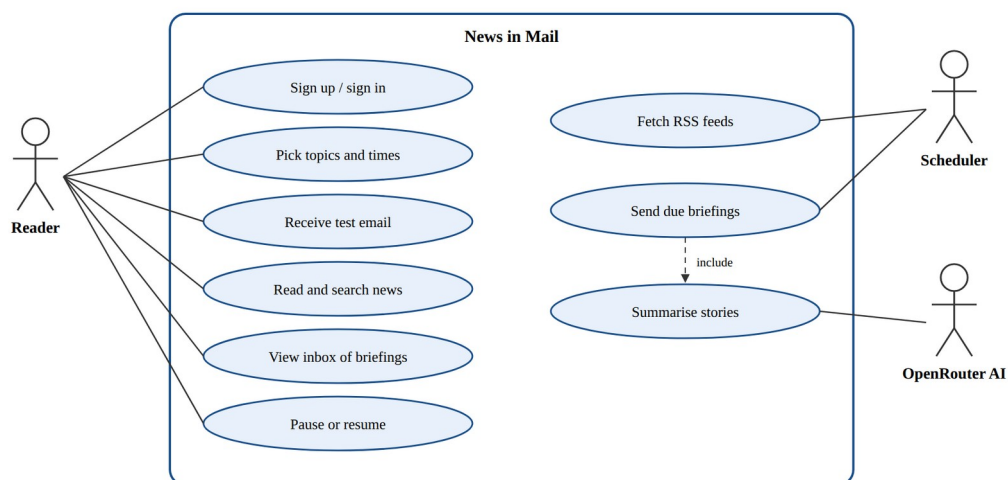


Figure 4.2: Use case diagram of News in Mail

4.8 SYSTEM DESIGN

4.8.1 System Architecture

Figure 4.3 shows the architecture. A single Next.js application serves the pages and runs the backend logic. A proxy file checks the session cookie before any private page loads. Pages are server components that read the database directly, and forms call server actions that validate input with Zod. All business rules live as pure functions in `src/lib`, and services in `src/server` talk to the database, the RSS feeds, OpenRouter and the mailer. A scheduler starts with the server and runs every five minutes.

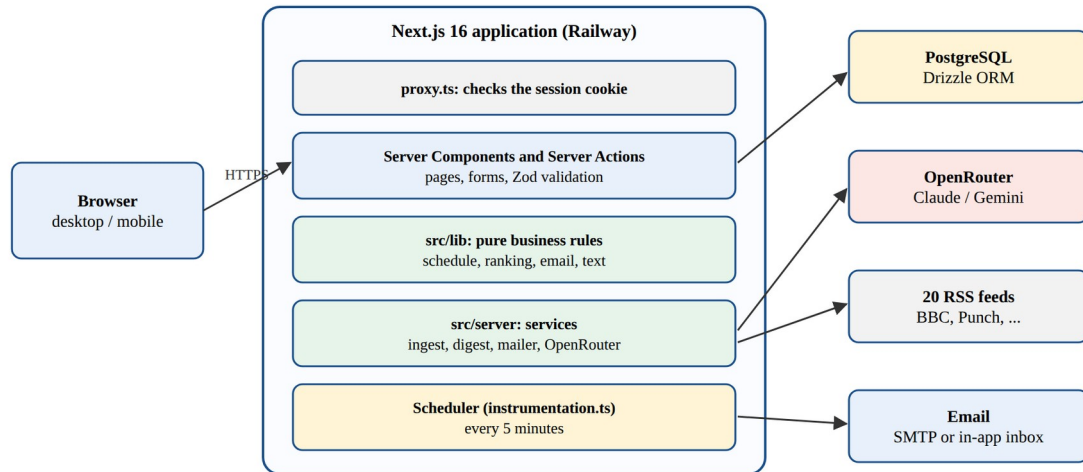


Figure 4.3: System architecture

4.8.2 The Briefing Process

Figure 4.4 shows what happens on each scheduler tick. A slot is due when it is switched on, its local time has passed today, it is less than three hours late and nothing has been recorded for it today. Stories are then chosen by a round-robin rule: only the reader's topics, only the last 36 hours, never a story sent in the last three days, at most three per topic and ten in total, so one busy topic cannot fill the email.

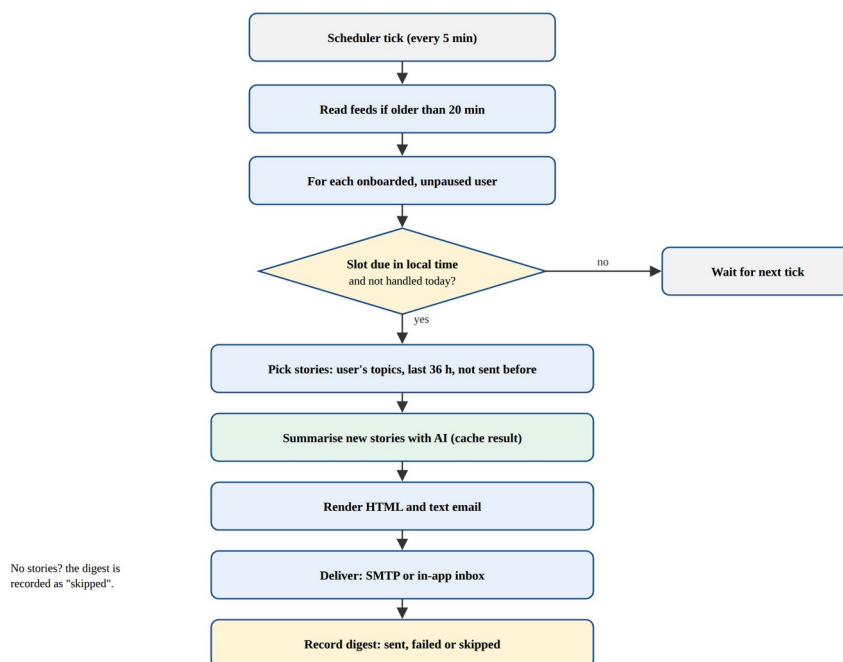


Figure 4.4: Flowchart of the briefing process

4.8.3 Sequence of the Test Email

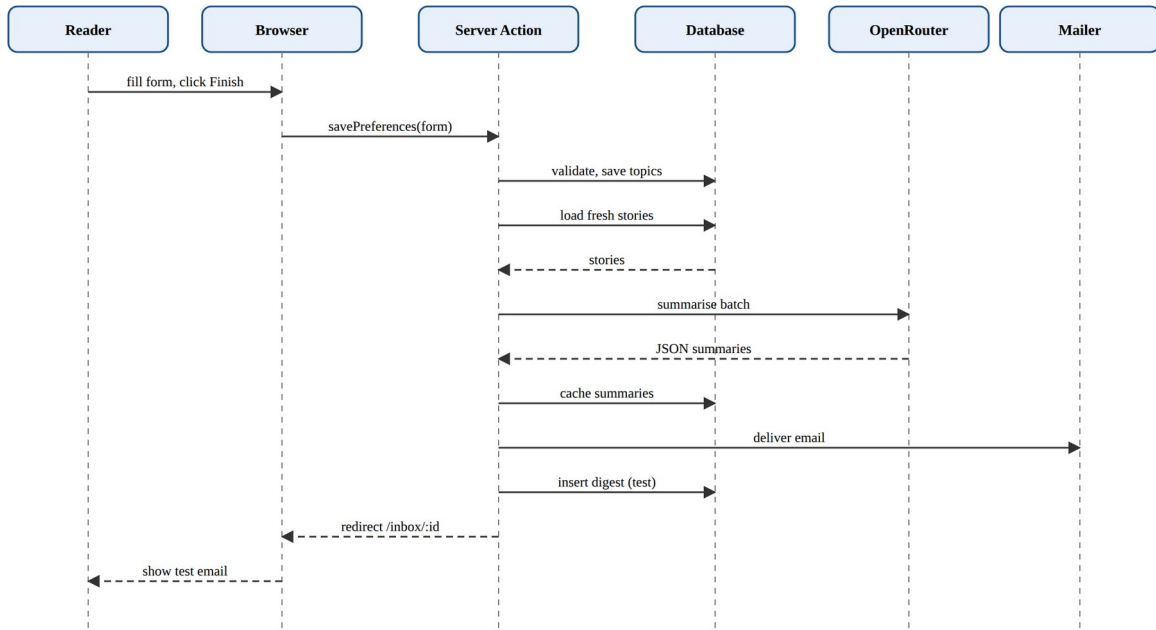


Figure 4.5: Sequence diagram for onboarding and the test email

4.8.4 Database Design

The database has four tables (Figure 4.6). The most important design decision is a unique index on (user_id, digest_date, slot) for morning and evening briefings. Even if two servers ran the scheduler at the same moment, the database would accept only one briefing per reader per slot per day.

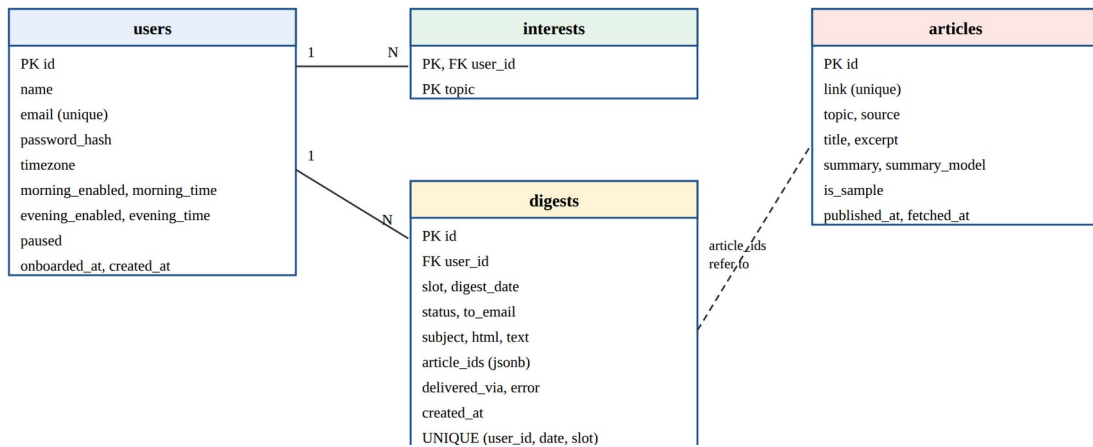


Figure 4.6: Entity relationship diagram

Table 4.3: Data dictionary of the digests table

Column	Type	Description
id	serial (PK)	Unique number of the briefing.
user_id	integer (FK)	Owner of the briefing; deleted with the user.
slot	text	morning, evening or test.

Column	Type	Description
digest_date	text	Local date (YYYY-MM-DD) in the reader's time zone.
status	text	sent, failed or skipped.
subject, html, text	text	The exact email that was produced.
article_ids	jsonb	Stories included, used to avoid repeats.
delivered_via, error	text	smtp or inbox, and the mail server error if any.

4.9 IMPLEMENTATION

Table 4.4: Tools and technologies used

Layer	Technology
Framework	Next.js 16 (App Router, Server Components, Server Actions), React 19, TypeScript in strict mode
Database	PostgreSQL with Drizzle ORM and drizzle-kit migrations
Validation and auth	Zod; bcryptjs; JWT signed with jose in an HTTP-only cookie
News and AI	rss-parser for 20 RSS feeds; OpenRouter (Claude Sonnet 5.5, Gemini Flash fallback)
Email	Nodemailer over SMTP, or the in-app inbox when no mail server is configured
Interface	Plain CSS, lucide-react icons, DiceBear avatars, Roboto font, custom SVG illustrations
Testing and hosting	Vitest, Playwright, GitHub and Railway

The core scheduling rule is a pure function that receives the current time as an argument, which makes it easy to test with a fixed clock:

```
export function dueSlots(at: Date, prefs: DeliveryPrefs, handledToday: Set<Slot>) {
  if (prefs.paused) return [];
  const { date, minutes } = localParts(at, prefs.timezone);
  return enabledSlots(prefs)
    .filter((s) => minutes >= s.minutes && minutes - s.minutes <= CATCH_UP_MINUTES)
    .filter((s) => !handledToday.has(s.slot))
    .map((s) => ({ slot: s.slot, date }));
}
```

The AI step sends all new stories of a briefing in one request and asks for JSON containing an introduction and one summary per story id. The reply is checked, typographic characters are replaced with plain ones, and each summary is saved on the story so it is reused for every reader. If the request fails, the first one or two sentences of the feed excerpt are used, so a briefing is never blocked by the AI service.

4.10 USER INTERFACE

The interface follows the clean style of Google News: a white header with a search bar, topic tabs underlined in blue, rounded story cards and a friendly greeting. Figures 4.7 to 4.14 show the main screens.

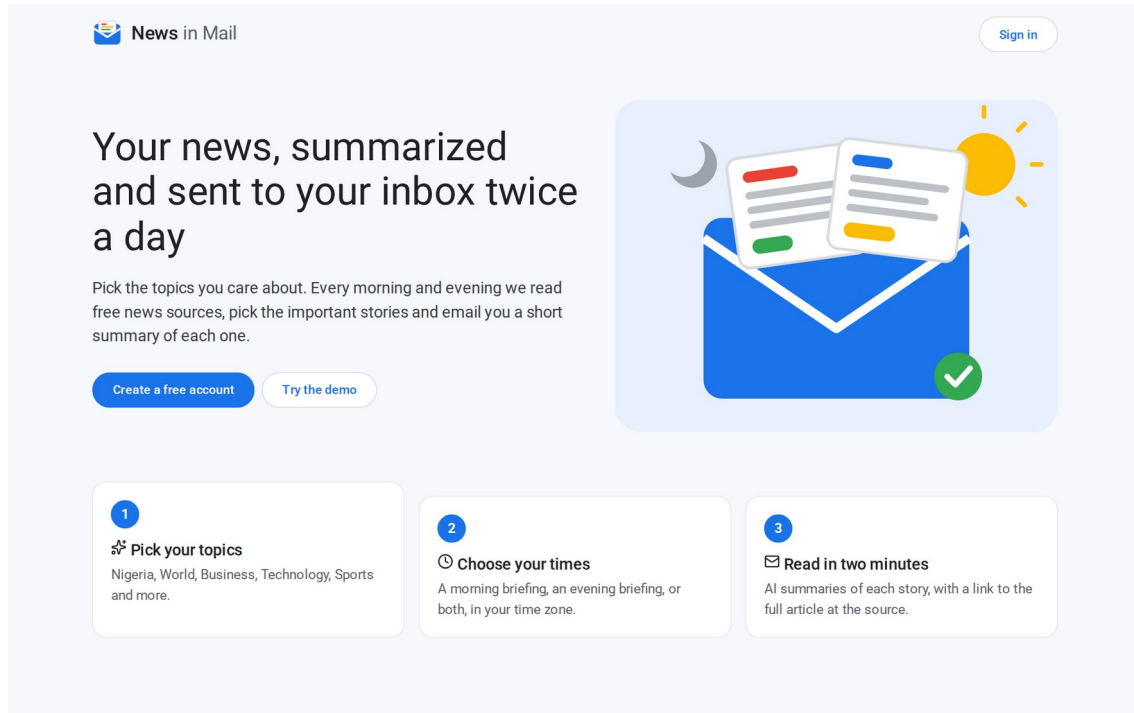


Figure 4.7: Landing page

News in Mail

Welcome, Godswill

Pick your topics and delivery times. When you finish we will send you a test email straight away so you can see what your briefings look like.

Topics
Pick the subjects you want in your briefings.

World ✓ Nigeria ✓ Africa
Business ✓ Technology ✓ Science
Health Sports Entertainment

Delivery times
We send at most two emails a day.

☒ **Morning briefing**
What happened overnight 7:00 AM

☒ **Evening briefing**
Catch up on the day 6:00 PM

Time zone
Africa/Lagos

[Finish and send test email](#)

Figure 4.8: Onboarding: picking topics and delivery times

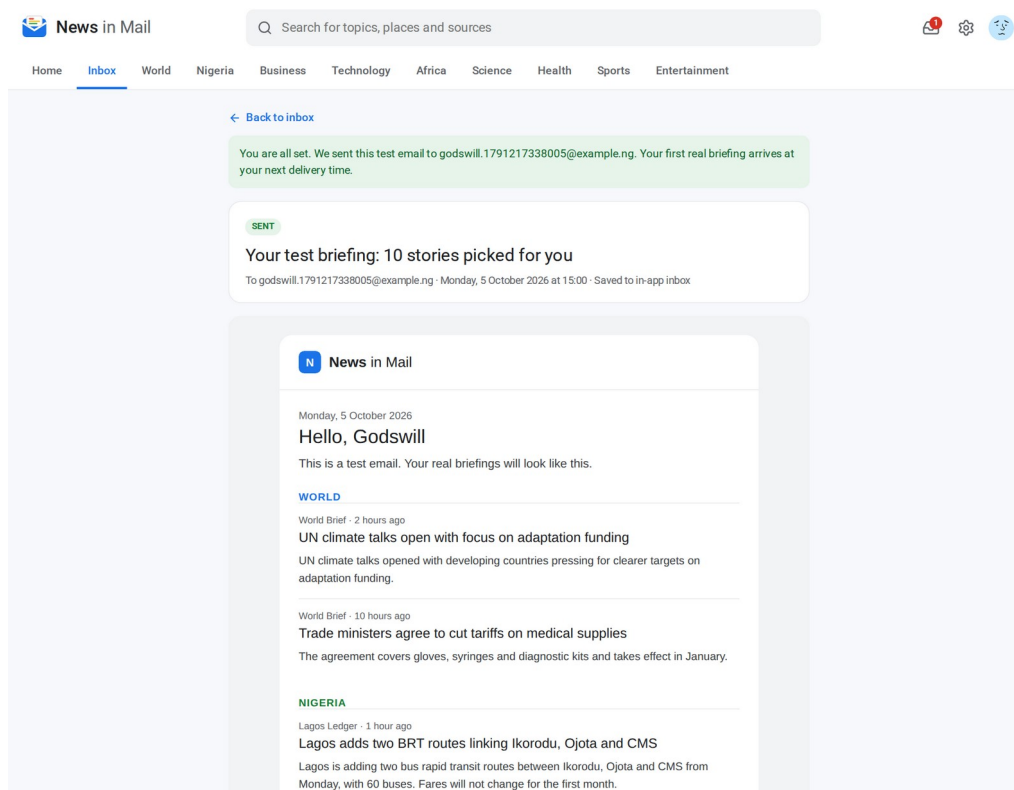


Figure 4.9: The test email shown right after onboarding

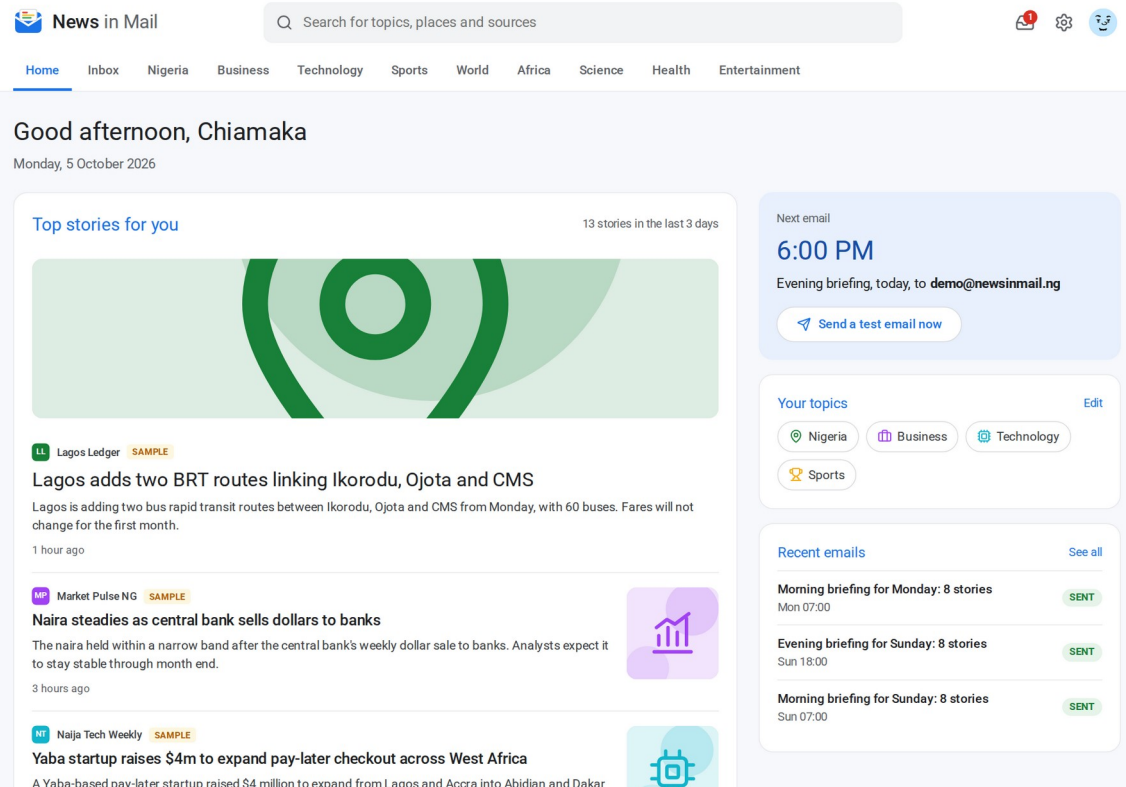


Figure 4.10: Home page with top stories, next email and recent emails

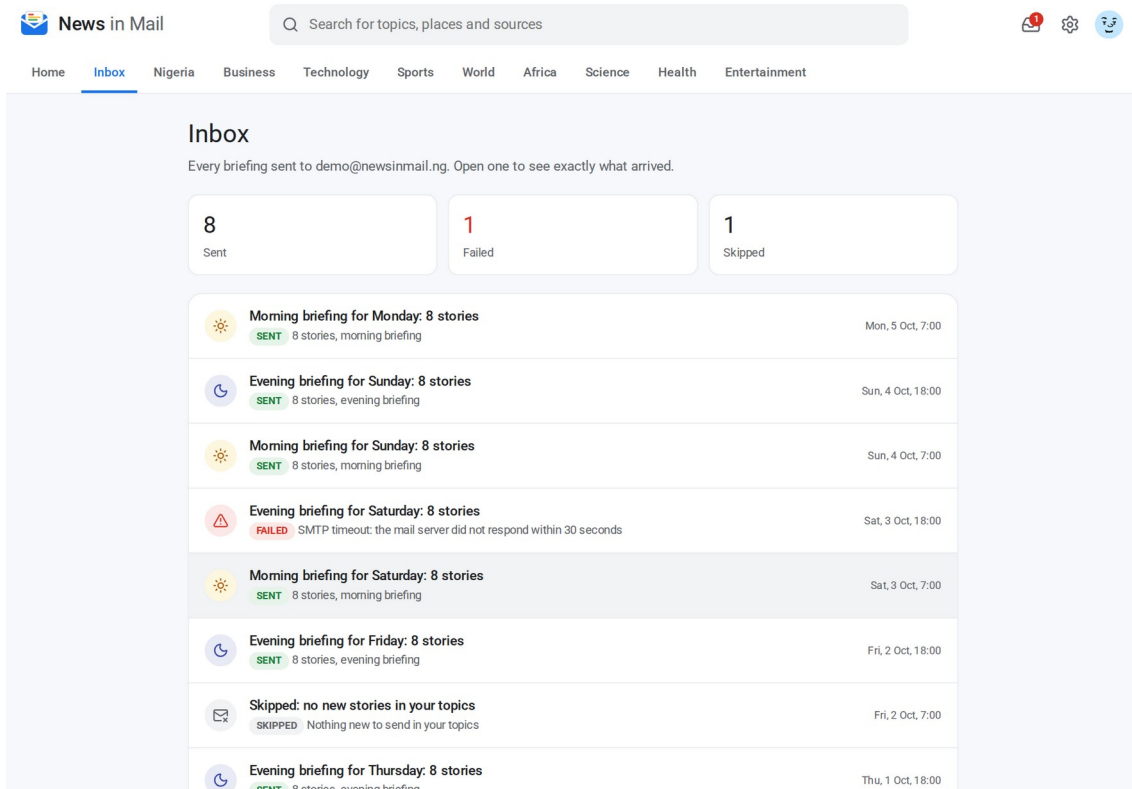


Figure 4.11: Inbox showing sent, failed and skipped briefings

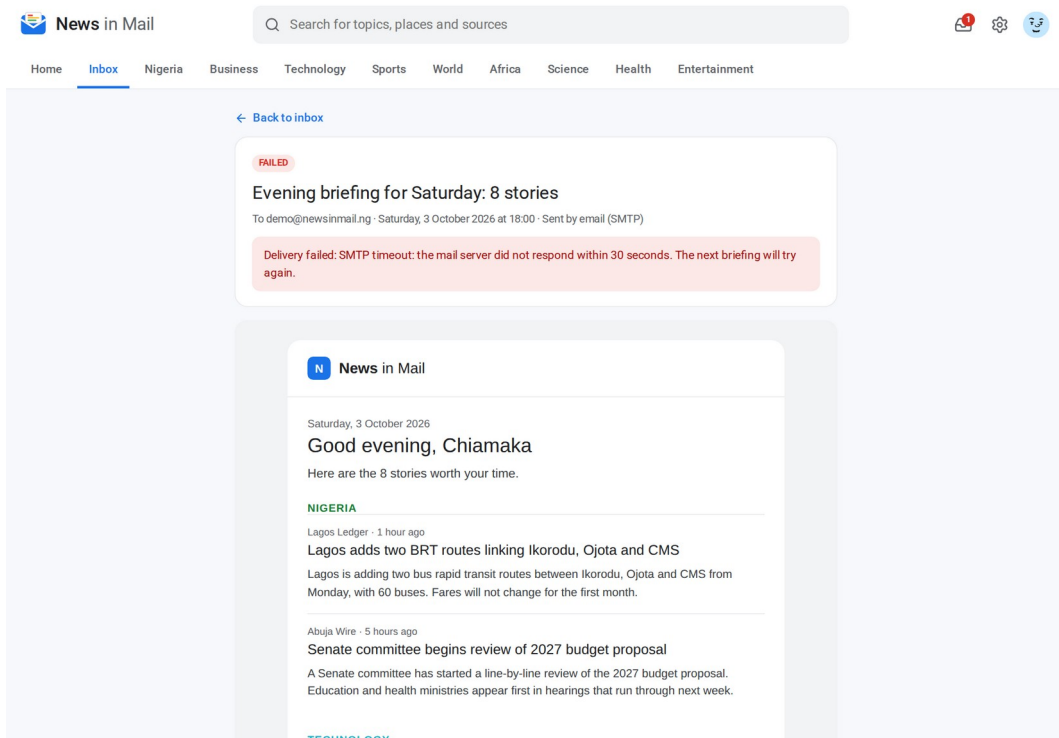


Figure 4.12: A failed briefing with the mail server error

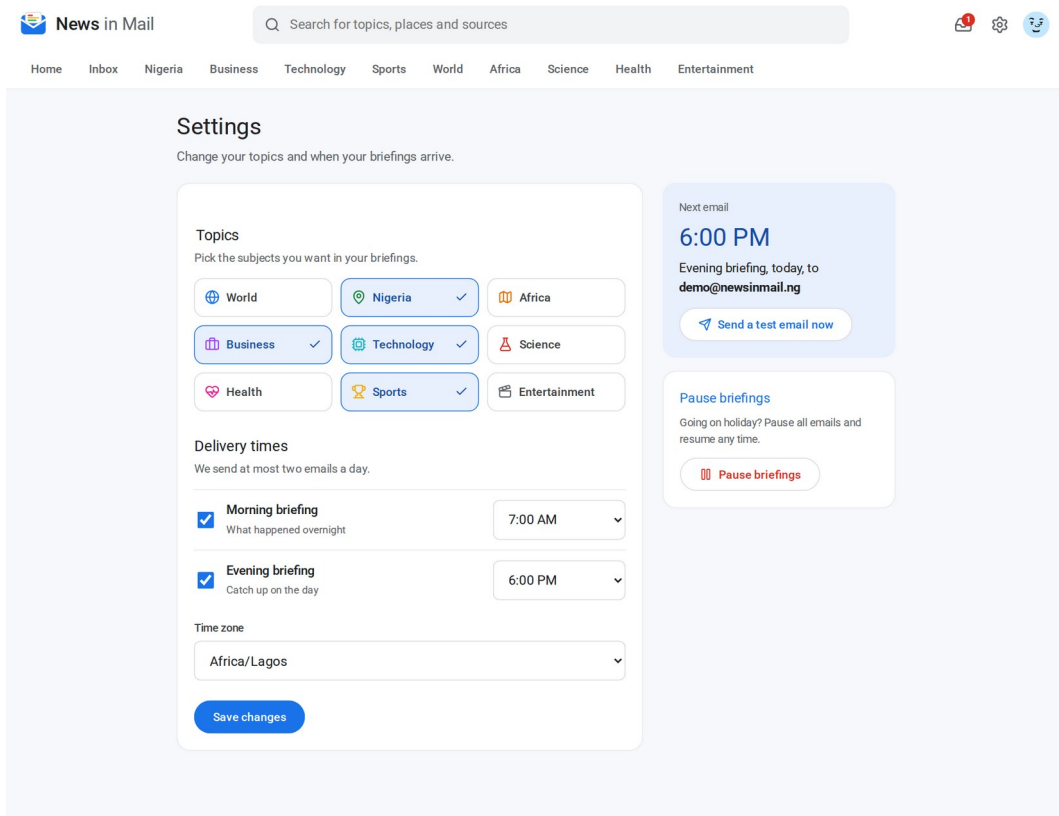


Figure 4.13: Settings, with pause and resume

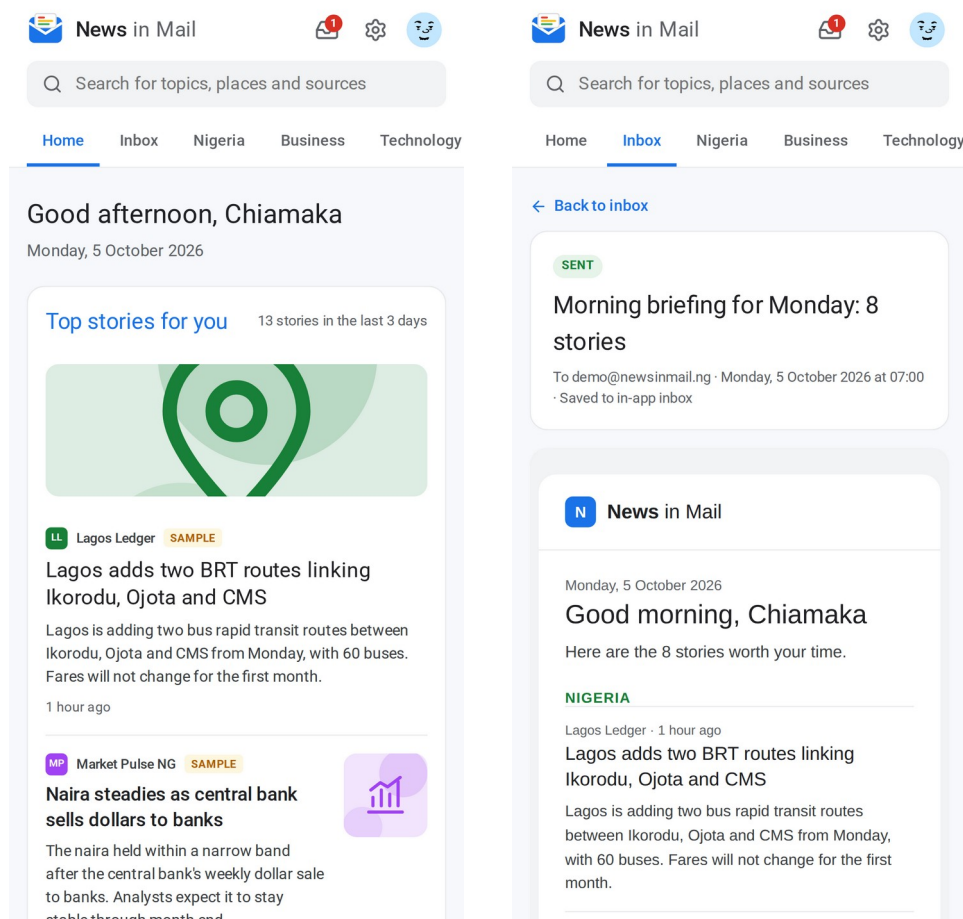


Figure 4.14: Mobile view: home (left) and an email (right)

4.11 TESTING

Testing was done at three levels. Unit tests check the pure business rules. Integration tests run the real services against a separate PostgreSQL test database. End-to-end tests drive a real browser through the application on desktop and on a Pixel 7 phone size. All tests use a fixed clock and run offline, so the results do not depend on the news of the day.

Table 4.5: Summary of automated tests

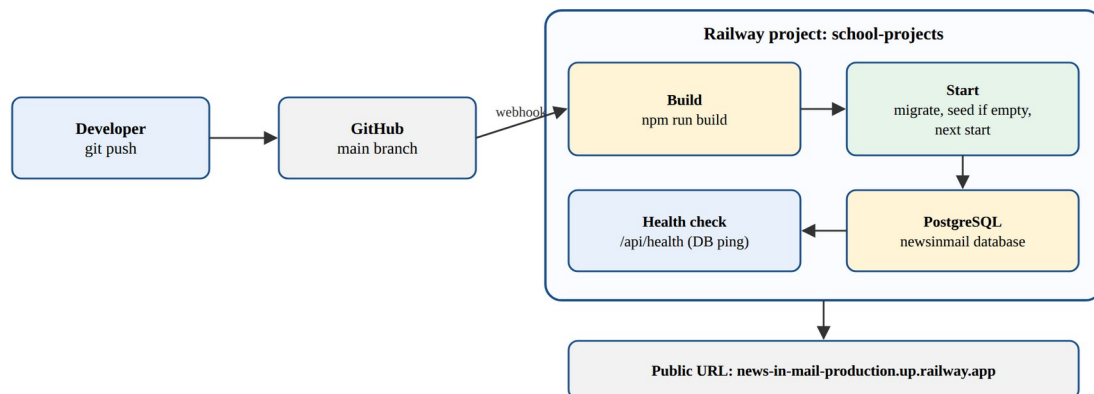
Suite	Tests	What it checks
Unit (Vitest)	52	Scheduling, story selection, text cleaning, email rendering, validation, sessions, AI reply parsing
Integration (Vitest)	9	Seeding, sending, skipping, no repeated stories, no duplicates, user scoping, RSS parsing
End-to-end desktop (Playwright)	14	Sign in and up, onboarding with test email, home, inbox, settings, search, access control
End-to-end mobile (Playwright)	2	No sideways scrolling on four pages; opening an email
Total	77	All passing

Table 4.6: Selected functional test cases

Test case	Expected result	Result
Sign in with the pre-filled demo account	Home page greets Chiamaka	Pass
Finish onboarding with no topic selected	Inline error: Pick at least one topic	Pass
Finish onboarding with two topics	Test email shown with only those topics	Pass
Run the scheduler twice at 6:05 PM	Only one evening briefing is created	Pass
Paused reader at briefing time	No briefing is sent	Pass
Open another reader's email by URL	Page not found (404)	Pass
View pages at phone width	No horizontal scrolling	Pass

4.12 DEPLOYMENT

The application is deployed on Railway and connected to the GitHub repository, so every merge to the main branch is built and deployed automatically (Figure 4.15). On start, the server applies database migrations, loads demo data only if the database is empty, and starts Next.js. Railway calls `/api/health`, which pings the database, and only sends traffic to the new version when it reports OK. Secrets such as the session key and the OpenRouter key are stored as environment variables, never in the code.

**Figure 4.15: Deployment pipeline on Railway**

4.13 RESULTS AND DISCUSSION

The deployed system is live at <https://news-in-mail-production.up.railway.app>. On its first run it read all 20 feeds without failure and stored 269 live stories. A new reader received a test email with AI summaries about eight seconds after finishing onboarding. The summaries were short, neutral and accurate to the source excerpts, and caching meant each story was summarised only once. All 77 automated tests pass, the health check confirms the database connection, and the pages work well on phones.

The main limitation is email delivery. No mail account was available during development, so briefings are stored in the in-app inbox. Real email is switched on by setting one SMTP variable, with no code change. Other limitations are that summaries depend on the quality of the feed excerpts and that the scheduler runs inside one server. Future work includes real email delivery with a verified domain, more Nigerian sources, a weekly digest option, and letting readers rate stories so the ranking can learn their preferences.

4.14 CONCLUSION

News in Mail meets all its objectives. It combines the skills I gained at Start Innovation Hub: database design, secure authentication, backend services, scheduled jobs, integration with an AI service, a responsive interface, automated testing and cloud deployment, all in one working product.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 SUMMARY

This report described my six months of SIWES at Start Innovation Hub, Uyo. It gave the history and objectives of SIWES and a profile of the hub, its services, programmes and impact. It then described my work in the Software Development unit: frontend foundations, backend APIs with Node.js, PostgreSQL databases, authentication and security, teamwork with Git, and two team projects. Finally it presented my mini project, News in Mail, a deployed web application that sends personalised, AI-summarised news briefings every morning and evening.

5.2 CONCLUSION

SIWES achieved its purpose for me. I now understand how professional software is planned, built, reviewed, tested and deployed, and I have real work to show for it. The hub's practical, project-based approach gave me both the technical skills and the confidence to keep learning on my own.

5.3 RECOMMENDATIONS

- To the ITF: pay student allowances on time and increase them to reflect current transport costs.
- To the University and Department: include more practical web, backend and database work, Git and cloud deployment in the curriculum before SIWES, and help students find placements early.
- To Start Innovation Hub: keep the structured learning track for interns and consider a small stipend or transport support for SIWES students.
- To future SIWES students: choose a placement that gives real work, keep your logbook up to date every day, ask questions and build a project you can show.

REFERENCES

- Industrial Training Fund (2023). Students Industrial Work Experience Scheme (SIWES): Information and Guidelines. Abuja: ITF.
- Start Innovation Hub (2026). Start Innovation Hub: Empowering Tech Talent and Startups in Nigeria. Retrieved from <https://starthub.com.ng>
- Start Innovation Hub (2026). Our Services. Retrieved from <https://starthub.com.ng/our-services>
- BusinessDay (2025, July 11). Start Innovation Hub commits N40m to bridge Nigeria's tech skill gap. Retrieved from <https://businessday.ng>
- Daily Trust (2025, January 24). Tech firm targets \$100million funding for startups in A/Ibom. Retrieved from <https://dailytrust.com>
- NLink (n.d.). Start Innovation Hub: Linking Partners for Niger Delta Development. Retrieved from <https://ndlink.org/start-innovation-hub>
- Techpoint Africa (n.d.). The Uyo tech scene. Retrieved from <https://techpoint.africa>
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. PhD thesis, University of California, Irvine.
- Sommerville, I. (2016). Software Engineering (10th ed.). Boston: Pearson.
- Vercel (2026). Next.js Documentation. Retrieved from <https://nextjs.org/docs>
- Drizzle Team (2026). Drizzle ORM Documentation. Retrieved from <https://orm.drizzle.team>
- PostgreSQL Global Development Group (2026). PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs>
- RSS Advisory Board (2009). RSS 2.0 Specification. Retrieved from <https://www.rssboard.org/rss-specification>
- Jones, M., Bradley, J. and Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force.
- OWASP Foundation (2021). OWASP Top 10 Web Application Security Risks. Retrieved from <https://owasp.org/Top10>
- OpenRouter (2026). OpenRouter API Documentation. Retrieved from <https://openrouter.ai/docs>

APPENDIX

LIVE SYSTEM AND SOURCE CODE

Live system: <https://news-in-mail-production.up.railway.app>

Demo login: `demo@newsinmail.ng`, password `demo1234` (filled in on the sign-in page).

Source code: available on request, with full instructions in the project README.

Run locally: start PostgreSQL, copy `.env.example` to `.env`, then run `npm install`, `npm run db:migrate`, `npm run db:seed` and `npm run dev`.